
DSC 40B - Hashing

Problem 1.

Suppose a hash table with 1000 bins stores 2000 numbers. Collisions are resolved with chaining.

True or False: it is possible for one of the hash table's bins to contain zero elements.

Problem 2.

Suppose a hash table is implemented so that it has ten bins, and that this number of bins is not increased when new elements are inserted; that is, the hash table is not allowed to "grow" or "resize". Let n be the number of elements stored in the hash table.

What is the expected time complexity of querying an element in this hash table, as a function of n ? You may assume that collisions are resolved with chaining, that the bins are linked lists, and that the hash function is "good" in that it appears to uniformly distribute elements among bins.

Problem 3.

What is the expected time complexity of the code below, assuming that *numbers* is a Python *set* of size n ? Recall that Python's sets are implemented as hash tables.

```
def foo(numbers):
    count = 0
    for x in numbers:
        if -x in numbers:
            count += 1
    return count
```

Problem 4.

Given two strings **a** and **b**, determine if they are isomorphic.

Two strings **a** and **b** are isomorphic if the characters in **a** can be replaced to get **b**. They must also have the same length.

Eg. "foo" and "bar" are not isomorphic, but "egg" and "add" are. "badc" and "baba" are not isomorphic.

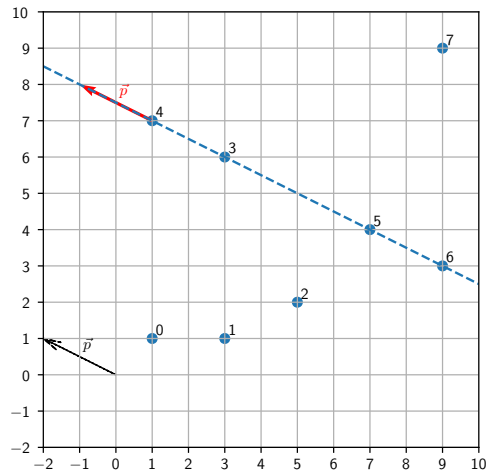
Describe an algorithm that determines if two strings are isomorphic.

Essentially, can the characters in **a** be mapped to the characters in **b** such that the characters in **a** can be replaced to get **b**.

Problem 5.

Consider the problem of finding collinear points. Suppose you are given a set X of n points in $\mathbb{R}^2$, along with a query vector $\vec{p} \in \mathbb{R}^2$. Your goal is to calculate the maximum number of points in X that fall on the same straight line parallel to $\vec{p}$. That is, considering all of the (infinitely) many lines parallel to $\vec{p}$, find the one that contains the most points from X .

For example, in the picture below, the black points are those in X and the arrow represents $\vec{p}$. The maximum number of points in X that falls on a line parallel to $\vec{p}$ is four (those on the dashed line).



In a file named `max_points_on_line.py`, write a function `max_points_on_line(X, p)` which accepts the following arguments:

- `X`: A list of points, each point being a 2-tuple of numbers.
- `p`: A query vector represented as a 2-tuple of numbers.

Your function should return the maximum number of points in `X` that fall on the same straight line parallel to $\vec{p}$. Your answer should be an integer. If `X` is empty, return 0. If a point $\vec{x}$ in `X` is such that $\vec{x} = \vec{p}$, you should consider to be on the same line as $\vec{p}$ (even if there are no other points).

Your algorithm should take expected time that is linear in the number of points in `X`.

Example (using the points in the picture above):

```
>>> X = [
... (1, 1),
... (3, 1),
... (5, 2),
... (3, 6),
... (1, 7),
... (7, 4),
... (9, 3),
... (9, 9)
... ]
>>> p = (-2, 1)
>>> max_points_on_line(X, p)
4
```

Hint: maybe you can use a hash table (i.e., a Python dictionary)? Remember, to use hashing in solving an algorithmic problem, you often need to turn the problem into a *membership query*. For any given point x , you can calculate the along $\vec{p}$ that passes through x . How can you turn checking to see if there are other points with the same slope into a membership query?